

Modul Praktikum Sistem Operasi



**Pengampu :
APRI SISWANTO, S.Kom, M.Kom, PhD**

**TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS ISLAM RIAU
PEKANBARU
2025/2026**

RPS PRAKTIKUM SISTEM OPERASI

Mata Kuliah	SISTEM OPERASI	
Bobot SKS	1 SKS	
	1 sks (P) = 170 Menit x 16 Minggu = 2720 Menit (45 Jam)	
Jumlah Modul	7 Modul	
	1. Command Prompt dan Powershell	6 Jam
	2. Introduction to linux kernel modules	6 Jam
	3. Proses	6 Jam
	4. Thread & Sinkronisasi	7 Jam
	5. Penjadwalan	7 Jam
	6. Deadlock	7 Jam
	7. Manajemen Memory (Main Memory)	7 Jam
Capaian Pembelajaran	Keterampilan Umum	KU9 : mampu mendokumentasikan, menyimpan, mengamankan, dan menemukan kembali data untuk menjamin kesahihan dan mencegah plagiasi;
	Keterampilan Khusus	KK4 : mampu merancang dan mengembangkan algoritma untuk keperluan <i>System & Network Administration</i> . KK5 : mampu mengidentifikasi dan menyelesaikan isu-isu / permasalahan yang muncul di masyarakat dengan pengetahuan dan keterampilan teknologi informasi terkini.

Daftar Isi

MODUL 1 : Command Prompt dan Powershell.....	1
1.1 Tujuan.....	1
1.2 Alat & Bahan.....	1
1.3 Dasar Teori.....	1
1.3.1 Command Line Interface.....	1
1.3.2 Command Prompt.....	1
1.3.3 Power Shell.....	2
1.4 Latihan.....	2
MODUL 2 : Introduction to linux kernel modules.....	3
2.1 Tujuan.....	3
2.2 Alat & Bahan.....	3
2.3 Dasar Teori.....	3
2.3.1 Apa itu Linux Kernel Module (LKM).....	3
2.3.2 Keuntungan Menggunakan LKM.....	3
2.3.3 Risiko.....	4
2.4 Persiapan Lingkungan.....	4
2.5 Langkah-Langkah Praktikum.....	5
2.5.1 Membuat Modul Kernel Sederhana.....	5
2.5.2 Menyusun File Makefile.....	6
2.5.3 Kompilasi modul.....	7
2.5.4 Memuat modul ke kernel.....	7
2.5.5 sudo rmmod hello.....	7
2.6 Latihan.....	8
MODUL 3 : Proses.....	10
3.1 Tujuan.....	10
3.2 Alat & Bahan.....	10
3.3 Dasar Teori.....	10
3.3.1 Fork().....	10
3.3.2 Exec.....	11
3.3.3 Wait.....	12
3.3.4 Sleep().....	14
3.3.5 Melihat Proses di Sistem.....	15
MODUL 4 : Thread & Sinkronisasi.....	15
4.1 Tujuan.....	16
4.2 Alat & Bahan.....	16
4.3 Dasar Teori.....	16
4.3.1 Thread.....	16
4.3.2 Sinkronisasi.....	16
4.3.3 Pengimplementasian Threads.....	17
4.4 Latihan.....	20
MODUL 5 : Penjadwalan.....	21
5.1 Tujuan.....	21

5.2 Alat & Bahan.....	21
5.3 Dasar Teori.....	21
5.3.1 Konsep Proses dalam Sistem Operasi.....	21
5.3.2 Penjadwalan Proses.....	21
5.3.3 Kriteria Penjadwalan.....	21
5.3.4 Jenis-Jenis Algoritma Penjadwalan.....	22
5.3.5 Peran Penjadwalan dalam Sistem Operasi.....	22
5.4 Latihan.....	22
MODUL 6 : Deadlock.....	24
6.1 Tujuan.....	25
6.2 Alat & Bahan.....	25
6.3 Dasar Teori.....	25
6.3.1. Penjelasan Konsep Deadlock.....	25
6.3.2 Mengimplementasikan Deadlock dalam Java.....	25
6.4 Latihan.....	29
MODUL 7 : Main Memory.....	30
7.1 Tujuan.....	30
7.3 Dasar Teori.....	31
7.3.1 Ruang Alamat Logika Dan Fisik.....	31
7.3.2 Mekanisme Paging.....	31
7.3.3 Contiguous Memory Allocation.....	34
7.3.4 Fragmentasi dan Kompaksi.....	35

MODUL 1 : Command Prompt dan Powershell

Kegiatan Belajar	Waktu (Jam) Belajar Mhs	Waktu (Jam) Dosen	Waktu (Jam) Asisten/Laboran
Tutorial / Pre- Test	0.5 Jam	Tidak hadir	Hadir
Persiapan	0.5 Jam	Tidak hadir	Hadir
Pelaksanaan Praktikum	3 Jam	Tidak hadir / Hadir	Hadir
Penyusunan Laporan	1.5 Jam	Tidak hadir	Hadir
Presentasi	0.5 Jam	Hadir	Hadir
Total Waktu (Menit)	6 Jam	0.5 Jam	5.5 Jam

1.1 Tujuan

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu memahami konsep dasar *Command Line Interface* (CLI) serta dapat menggunakan Command Prompt dan PowerShell untuk melakukan pengelolaan file dan direktori pada sistem operasi. Selain itu, mahasiswa diharapkan mampu mengimplementasikan perintah-perintah dasar Command Prompt dan PowerShell dalam menyelesaikan studi kasus pengelolaan sistem secara sederhana.

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat memahami langkah-langkah dalam Bayesian Learning dan dapat mengimplementasikan ke dalam bahasa pemrograman yang diinginkan.

1.2 Alat & Bahan

Alat & bahan yang digunakan adalah hardware perangkat PC beserta kelengkapannya berjumlah 40 PC dengan aplikasi yang diinginkan dalam pembuatan program.

1.3 Dasar Teori

1.3.1 Command Line Interface

Command Line Interface (CLI) merupakan antarmuka berbasis teks yang digunakan untuk berinteraksi langsung dengan sistem operasi melalui perintah-perintah tertentu. CLI memungkinkan pengguna menjalankan perintah secara lebih efisien, cepat, dan fleksibel dibandingkan antarmuka grafis (*Graphical User Interface*). Melalui CLI, pengguna dapat melakukan pengelolaan file, direktori, serta konfigurasi sistem dengan tingkat kontrol yang lebih mendalam.

1.3.2 Command Prompt

Command Prompt merupakan salah satu aplikasi CLI pada sistem operasi Windows yang digunakan untuk menjalankan perintah-perintah dasar sistem. Command Prompt berfokus pada operasi sederhana seperti menampilkan isi direktori, berpindah folder, membuat dan menghapus file maupun folder, serta melakukan pengelolaan file dasar. Beberapa perintah penting pada Command Prompt antara lain:

- `dir` - Menampilkan isi direktori saat ini.
- `cd` - Berpindah direktori.
- `mkdir` - Membuat direktori baru.
- `rmdir` - Menghapus direktori.

- del - Menghapus file.
- copy - Menyalin file dari satu lokasi ke lokasi lain.
- move - Memindahkan file ke lokasi lain.

Penggunaan command prompt

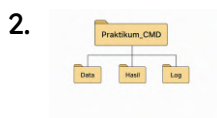
- Melihat tanggal, waktu, dan versi sistem.
- Menampilkan isi folder.
- Menampilkan isi folder dengan berbagai format tampilan.
- Berpindah ke direktori lain.
- Membuat folder baru.
- Masuk ke dalam folder.
- Membuat file teks baru.
- Menyalin file ke lokasi lain.
- Memindahkan file ke folder lain.
- Menghapus file.
- Menghapus folder beserta isinya.
- Membersihkan tampilan layar CMD.

1.3.3 Power Shell

PowerShell adalah CLI yang lebih canggih dibandingkan Command Prompt karena mendukung *scripting* dan otomatisasi tugas berbasis objek serta terintegrasi dengan .NET Framework. PowerShell memungkinkan pengguna untuk melakukan pengelolaan sistem yang lebih kompleks, seperti manipulasi file, manajemen proses, dan administrasi sistem secara otomatis. PowerShell juga mendukung penggunaan *cmdlet* yang mempermudah penulisan perintah secara terstruktur.

1.4 Latihan

1. Buat struktur folder berikut menggunakan CMD:



3. Buat file teks di dalam folder **Data** berisi biodata singkat Anda.
4. Salin file tersebut ke folder **Hasil** dan pindahkan **Folder** hasil ke folder Log.
5. Hapus file asli dari folder Data.

MODUL 2 : Introduction to linux kernel modules

Kegiatan Belajar	Waktu (Jam) Belajar Mhs	Waktu (Jam) Dosen	Waktu (Jam) Asisten/Laboran
Tutorial / Pre- Test	0.5 Jam	Tidak hadir	Hadir
Persiapan	0.5 Jam	Tidak hadir	Hadir
Pelaksanaan Praktikum	3 Jam	Tidak hadir / Hadir	Hadir
Penyusunan Laporan	1.5 Jam	Tidak hadir	Hadir
Presentasi	0.5 Jam	Hadir	Hadir
Total Waktu (Menit)	6 Jam	0.5 Jam	5.5 Jam

2.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep dasar Linux Kernel Module (LKM) serta bagaimana modul tersebut bekerja menambah fungsi pada kernel Linux.
2. Mampu menulis, meng-compile, dan memuat (*load*) kode program modul kernel sederhana ke dalam sistem operasi.
3. Mampu mengimplementasikan interaksi modul melalui filesystem /proc serta melakukan penghapusan (*unload*) modul secara aman.

2.2 Alat & Bahan

Alat & bahan yang digunakan adalah hardware perangkat PC/Laptop beserta kelengkapannya berjumlah 40 PC/Laptop dengan software aplikasi yang diinginkan dalam pembuatan program.

2.3 Dasar Teori

2.3.1 Apa itu Linux Kernel Module (LKM)

Kernel module adalah potongan kode yang dapat dimuat ke dalam kernel Linux saat sistem berjalan (tanpa perlu *reboot*).

Modul ini menambah fungsionalitas kernel, seperti dukungan untuk perangkat keras, sistem file, atau fitur tambahan lainnya.

Analogi sederhana:

Kernel = mesin utama OS

Modul kernel = *plugin* yang bisa ditambah atau dicabut tanpa mematikan sistem.

2.3.2 Keuntungan Menggunakan LKM

- Tidak perlu *reboot* untuk menambah fitur baru.
- Memungkinkan pengembangan dan pengujian kernel dengan risiko rendah.
- Cocok untuk pembelajaran dan eksperimen sistem operasi.

2.3.3 Risiko

Kode kernel berjalan dengan hak akses tertinggi.

Kesalahan kecil dapat menyebabkan sistem crash atau hang.

Oleh karena itu, selalu gunakan Virtual Machine (VM).

2.4 Persiapan Lingkungan

1. Gunakan Linux Virtual Machine (Ubuntu 20.04 atau sejenisnya).
2. Pastikan telah terinstal:
3. `sudo apt update`

```
apri@apri-UIR:~$ sudo su
[sudo] password for apri:
root@apri-UIR:/home/apri# sudo apt update
Ign:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Ign:2 http://id.archive.ubuntu.com/ubuntu jammy InRelease
Ign:3 http://id.archive.ubuntu.com/ubuntu jammy-updates InRelease
Ign:4 http://id.archive.ubuntu.com/ubuntu jammy-backports InRelease
Ign:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Ign:2 http://id.archive.ubuntu.com/ubuntu jammy InRelease
Ign:3 http://id.archive.ubuntu.com/ubuntu jammy-updates InRelease
```

4. `sudo apt install build-essential linux-headers-$(uname -r)`

```
0.0.0-47-generic
root@apri-UIR:/home/apri# sudo apt install build-essential linux-headers-$(uname -r)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

5. Siapkan direktori kerja:
6. `mkdir ~/kernel_module_praktikum`

```
root@apri-UIR:/home/apri# mkdir kernel_module_praktikum
```

7. `cd ~/kernel_module_praktikum`

```
root@apri-UIR:/home/apri# cd kernel_module_praktikum/
root@apri-UIR:/home/apri/kernel_module_praktikum#
```

2.5 Langkah-Langkah Praktikum

2.5.1 Membuat Modul Kernel Sederhana

Buat file baru bernama hello.c:

nano hello.c

```
root@apri-UIR:/home/apri/kernel_module_praktikum# nano hello.c
```

Isi dengan kode berikut:

```
#include <linux/init.h>

#include <linux/module.h>

#include <linux/kernel.h>

MODULE_LICENSE("GPL");

MODULE_AUTHOR("Nama Anda");

MODULE_DESCRIPTION("Modul Kernel Sederhana");

MODULE_VERSION("1.0");

static int __init hello_init(void) {

    printk(KERN_INFO "Hello, Kernel Module Loaded!\n");

    return 0;

}

static void __exit hello_exit(void) {

    printk(KERN_INFO "Goodbye, Kernel Module Removed!\n");

}

module_init(hello_init);

module_exit(hello_exit);
```

```
GNU nano 6.2 hello.c
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Nama Anda");
MODULE_DESCRIPTION("Modul Kernel Sederhana");
MODULE_VERSION("1.0");
static int __init hello_init(void) {
    printk(KERN_INFO "Hello, Kernel Module Loaded!\n");
    return 0;
}
static void __exit hello_exit(void) {
    printk(KERN_INFO "Goodbye, Kernel Module Removed!\n");
}
module_init(hello_init);
module_exit(hello_exit);
```

Penjelasan singkat:

- `printk()` → setara dengan `printf()`, tapi untuk *kernel log*.
- `__init` dan `__exit` → menandai fungsi yang dijalankan saat modul dimuat/dihapus.

2.5.2 Menyusun File Makefile

Masih di folder yang sama, buat file bernama Makefile:

nano Makefile

```
root@apri-UIR: /home/apri/kernel_module_praktikum# nano Makefile
```

Isi dengan:

obj-m += hello.o

all:

make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD) modules

clean:

make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD) clean

```
GNU nano 6.2 Makefile
obj-m += hello.o
all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

2.5.3 Kompilasi modul

Jalankan perintah:

Make

```
root@apri-UIR:/home/apri/kernel_module_praktikum# make
```

Jika berhasil, akan muncul file hello.ko (kernel object file).

```
root@apri-UIR:/home/apri/kernel_module_praktikum# ls
hello.c  hello.mod  hello.mod.o  Makefile  Module.symvers
hello.ko hello.mod.c hello.o    modules.order
root@apri-UIR:/home/apri/kernel_module_praktikum#
```

2.5.4 Memuat modul ke kernel

sudo insmod hello.ko

Cek apakah modul sudah terpasang:

lsmod | grep hello

Untuk melihat pesan kernel:

dmesg | tail

Maka akan muncul Hello, Kernel Module Loaded!

```
root@apri-UIR:/home/apri/kernel_module_praktikum# sudo rmmod hello
root@apri-UIR:/home/apri/kernel_module_praktikum# dmesg | tail
[ 64.152120] audit: type=1107 audit(1761349544.168:62): pid=589 uid=102 auid=4294967295 ses=4294967295 subj=unconfined msg='apparmor="DENIED" operation="dbus_method_call" bus="system" path="/org/freedesktop/PolicyKit1/Authority" interface="org.freedesktop.DBus.Properties" member="GetAll" mask="send" name=":1.5" pid=1722 label="snap.snap-store.ubuntu-software" peer_pid=614 peer_label="unconfined"
exe="/usr/bin/dbus-daemon" sauid=102 hostname=? addr=? terminal=?'
[ 64.153068] audit: type=1107 audit(1761349544.170:63): pid=589 uid=102 auid=4294967295 ses=4294967295 subj=unconfined msg='apparmor="DENIED" operation="dbus_method_call" bus="system" path="/org/freedesktop/PolicyKit1/Authority" interface="org.freedesktop.PolicyKit1.Authority" member="CheckAuthorization" mask="send" name=":1.5" pid=1722 label="snap.snap-store.ubuntu-software" peer_pid=614 peer_label="unconfined"
exe="/usr/bin/dbus-daemon" sauid=102 hostname=? addr=? terminal=?'
[ 64.880704] audit: type=1400 audit(1761349544.897:64): apparmor="DENIED" operation="open" class="file" profile="snap.snap-store.ubuntu-software" name="/etc/appstream.conf" pid=1722 comm="snap-store" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 1082.599370] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 1038546885 wd_nsec: 1038546439
[ 1361.790030] hello: loading out-of-tree module taints kernel.
[ 1361.790039] hello: module verification failed: signature and/or required key missing - tainting kernel
[ 1361.790934] Hello, Kernel Module Loaded!
```

2.5.5 sudo rmmod hello

dmesg | tail

Perhatikan pesan *Goodbye, Kernel Module Removed!* di log kernel.

```

root@apri-UIR:/home/apri/kernel_module_praktikum# sudo rmmod hello
root@apri-UIR:/home/apri/kernel_module_praktikum# dmesg | tail
[ 64.152120] audit: type=1107 audit(1761349544.168:62): pid=589 uid=102 auid=4294967295 ses=4294967295 subj=unconfined msg='apparmor="DENIED" operation="dbus_method_call" bus="system" path="/org/freedesktop/PolicyKit1/Authority" interface="org.freedesktop.PolicyKit1.Authority" member="CheckAuthorization" mask="send" name=":1.5" pid=1722 label="snap.snap-store.ubuntu-software" peer_pid=614 peer_label="unconfined"
exe="/usr/bin/dbus-daemon" sauid=102 hostname=? addr=? terminal=?'
[ 64.153068] audit: type=1107 audit(1761349544.170:63): pid=589 uid=102 auid=4294967295 ses=4294967295 subj=unconfined msg='apparmor="DENIED" operation="dbus_method_call" bus="system" path="/org/freedesktop/PolicyKit1/Authority" interface="org.freedesktop.PolicyKit1.Authority" member="CheckAuthorization" mask="send" name=":1.5" pid=1722 label="snap.snap-store.ubuntu-software" peer_pid=614 peer_label="unconfined"
exe="/usr/bin/dbus-daemon" sauid=102 hostname=? addr=? terminal=?'
[ 64.880704] audit: type=1400 audit(1761349544.897:64): apparmor="DENIED" operation="open" class="file" profile="snap.snap-store.ubuntu-software" name="/etc/appstream.conf" pid=1722 comm="snap-store" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 1082.599370] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 1038546885 wd_nsec: 1038546439
[ 1361.790030] hello: loading out-of-tree module taints kernel.
[ 1361.790039] hello: module verification failed: signature and/or required key missing - tainting kernel
[ 1361.790934] Hello, Kernel Module Loaded!
[ 1482.021552] Goodbye, Kernel Module Removed!

```

2.6 Latihan

Membuat Modul dengan Entry di /proc

1. Salin hello.c menjadi proc_hello.c.
2. Ubah kodenya agar membuat file di /proc.

Contoh:

```
#include <linux/init.h>
```

```
#include <linux/module.h>
```

```
#include <linux/kernel.h>
```

```
#include <linux/proc_fs.h>
```

```
#include <linux/seq_file.h>
```

```
MODULE_LICENSE("GPL");
```

```
MODULE_AUTHOR("Nama Anda");
```

```
MODULE_DESCRIPTION("Modul Kernel dengan /proc");
```

```
MODULE_VERSION("1.0");
```

```
static int proc_show(struct seq_file *m, void *v) {
```

```
    seq_printf(m, "Hello from /proc/myhello!\n");
```

```
    return 0;
```

```
}
```

```
static int proc_open(struct inode *inode, struct file *file) {
```

```

        return single_open(file, proc_show, NULL);
    }

static const struct proc_ops proc_file_ops = {
    .proc_open = proc_open,
    .proc_read = seq_read,
};

static int __init proc_init(void) {
    proc_create("myhello", 0, NULL, &proc_file_ops);
    printk(KERN_INFO "proc_hello module loaded.\n");
    return 0;
}

static void __exit proc_exit(void) {
    remove_proc_entry("myhello", NULL);
    printk(KERN_INFO "proc_hello module unloaded.\n");
}

module_init(proc_init);
module_exit(proc_exit);

Setelah dikompilasi dan dimuat, periksa:

cat /proc/myhello

```

MODUL 3 : Proses

Kegiatan Belajar	Waktu (Jam) Belajar Mhs	Waktu (Jam) Dosen	Waktu (Jam) Asisten/Laboran
Tutorial / Pre- Test	0.5 Jam	Tidak hadir	Hadir
Persiapan	0.5 Jam	Tidak hadir	Hadir
Pelaksanaan Praktikum	3 Jam	Tidak hadir / Hadir	Hadir
Penyusunan Laporan	1.5 Jam	Tidak hadir	Hadir
Presentasi	0.5 Jam	Hadir	Hadir
Total Waktu (Menit)	6 Jam	0.5 Jam	5.5 Jam

3.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Menjelaskan konsep dasar proses dalam sistem operasi.
2. Mengamati cara pembuatan proses baru menggunakan fork().
3. Menjelaskan mekanisme eksekusi program menggunakan exec().
4. Menjelaskan hubungan antara proses induk dan anak.
5. Menganalisis status proses dengan perintah Linux (ps, top).

3.2 Alat & Bahan

Alat & bahan yang digunakan adalah hardware perangkat PC beserta kelengkapannya berjumlah 40 PC dengan software aplikasi yang diinginkan dalam pembuatan program.

3.3 Dasar Teori

3.3.1 Fork()

Fork adalah mekanisme pada sistem operasi (khususnya OS berbasis UNIX/Linux) untuk membuat proses baru dengan cara menggandakan proses yang sedang berjalan (parent process) menjadi child process. Pada saat sebuah proses memanggil fork():

- Fork membuat salinan hampir identik dari proses.
- Proses asli disebut parent process.
- Proses baru disebut child process.

Berikut adalah langkah langkah praktikum:

1. Ketikkan **sudo su** dan isi dengan password login ubuntu seperti biasa.
2. Buat folder **praktikum3** dengan perintah **mkdir praktikum3**.
3. Masuk ke folder **praktikum3** dengan perintah **cd praktikum3**.
4. Buat proses (File bahasa C) dengan mengetikkan perintah **nano proses1.c**
5. Isi codingan **proses1.c** sesuai gambar di bawah ini. Pastikan codingan sesuai dengan gambar di bawah ini. Perhatikan simbol, besar dan kecilnya huruf.

```
root@azrirafl03: /home/azrirafl03/praktikum3
GNU nano 6.2 proses1.c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if(pid < 0) {
        fprintf(stderr, "Fork gagal\n");
        return 1;
    } else if(pid == 0) {
        printf("Ini proses anak dengan PID: %d\n", getpid());
    } else {
        printf("Ini proses induk dengan PID: %d\n", getpid());
    }

    return 0;
}
```

6. Simpan sebagai **proses1.c** dengan menekan **CTRL + X**, lalu tekan **y**, dan lalu Enter.
7. Lakukan kompilasi file **proses1.c** menjadi **proses1**. Perintahnya pada bagian b).
 - a. Format perintah : **gcc nama_file.c -o nama_file**.
 - b. **gcc proses1.c -o proses1**.
8. Jalankan codingan atau file **proses1.c** dengan mengetik perintah **./proses1**
9. Outputnya akan menjadi seperti gambar di bawah ini.

```
root@azrirafl03: /home/azrirafl03/praktikum3
root@azrirafl03:/home/azrirafl03/praktikum3# nano proses1.c
root@azrirafl03:/home/azrirafl03/praktikum3# gcc proses1.c -o proses1
root@azrirafl03:/home/azrirafl03/praktikum3# ./proses1
Ini proses induk dengan PID: 2133
Ini proses anak dengan PID: 2134
root@azrirafl03:/home/azrirafl03/praktikum3#
```

10. Analisis Codingan dan hasil dari **proses1.c** sebagai berikut.
 - a. Dua output akan muncul, satu dari induk, satu dari anak.
 - b. Urutan bisa berbeda karena eksekusi paralel.
 - c. Ketika fungsi **fork()** dipanggil dan di input ke variabel **pid**, terdapat nilai **pid** yang berbeda untuk tiap proses.
 - i. Proses Induk. **pid = nilai_pid_anak** (nilai integer > 0).
 - ii. Proses Anak dengan **pid = 0**.
 - iii. Gagal membuat proses dengan **pid = nilai_negatif**.
 - d. Cara kerja **fork()**, yaitu dengan membuat codingan menjalankan ulang program sebanyak 1x setelah pemanggilan fungsi **fork()**.

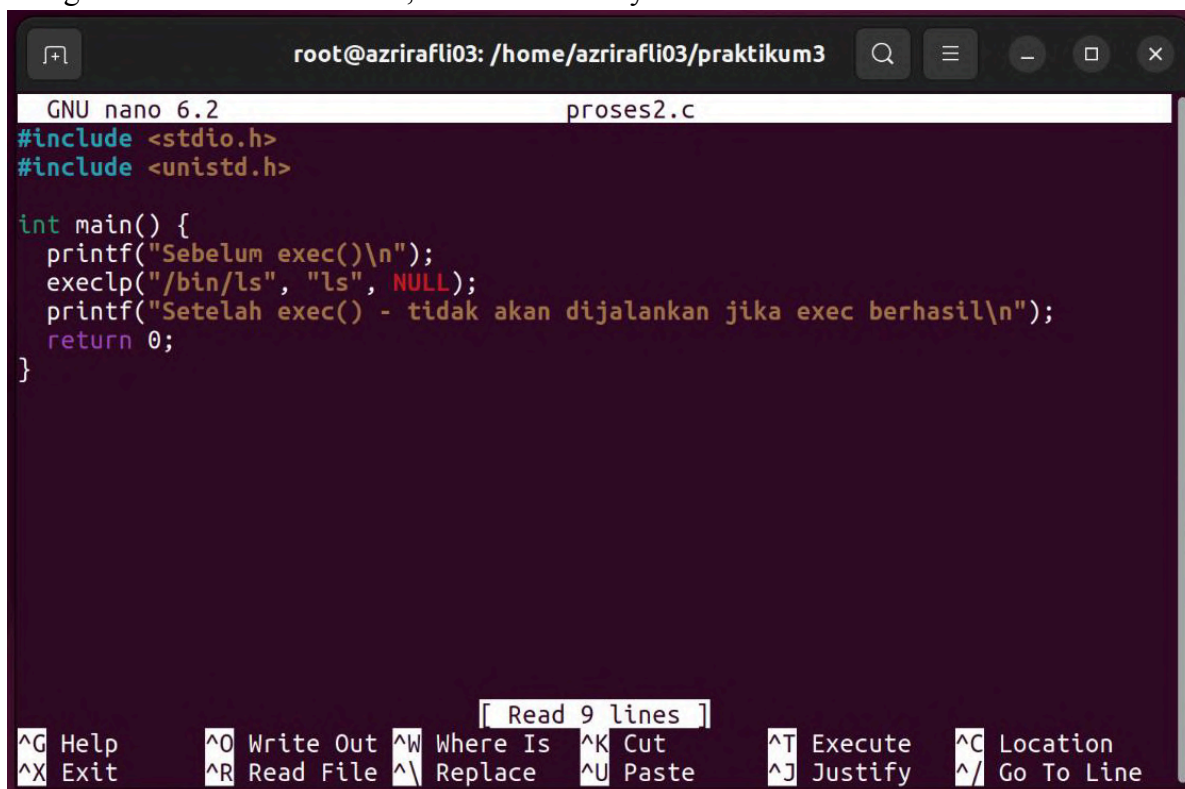
3.3.2 Exec

Exec adalah keluarga system call pada sistem operasi **UNIX/Linux** yang berfungsi untuk mengganti program yang sedang dijalankan oleh sebuah proses dengan program lain.

Berikut adalah langkah-langkah praktikum:

1. Buat proses untuk percobaan **exec()** dengan perintah **nano proses2.c**

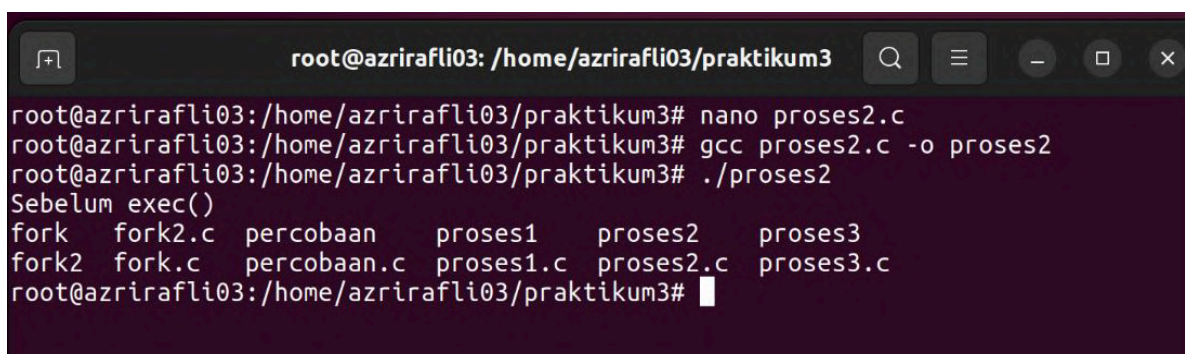
2. Isi codingan proses2.c sesuai gambar di bawah ini. Pastikan codingan sesuai dengan di gambar. Perhatikan simbol, besar dan kecilnya huruf.



```
GNU nano 6.2      proses2.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Sebelum exec()\n");
    execlp("/bin/ls", "ls", NULL);
    printf("Setelah exec() - tidak akan dijalankan jika exec berhasil\n");
    return 0;
}
```

3. Simpan sebagai proses2.c dengan menekan CTRL + X, lalu tekan y, dan lalu Enter.
4. Lakukan kompilasi file proses2.c menjadi proses2. Perintahnya pada bagian b).
- Format perintah : **gcc nama_file.c -o nama_file**
 - gcc proses2.c -o proses2**
5. Jalankan codingan atau file **proses2.c** dengan mengetik perintah **./proses2**.
6. Outputnya akan seperti pada gambar di bawah ini.



```
root@azirafli03: /home/azirafli03/praktikum3
root@azirafli03:/home/azirafli03/praktikum3# nano proses2.c
root@azirafli03:/home/azirafli03/praktikum3# gcc proses2.c -o proses2
root@azirafli03:/home/azirafli03/praktikum3# ./proses2
Sebelum exec()
fork  fork2.c  percobaan  proses1  proses2  proses3
fork2  fork.c  percobaan.c  proses1.c  proses2.c  proses3.c
root@azirafli03:/home/azirafli03/praktikum3#
```

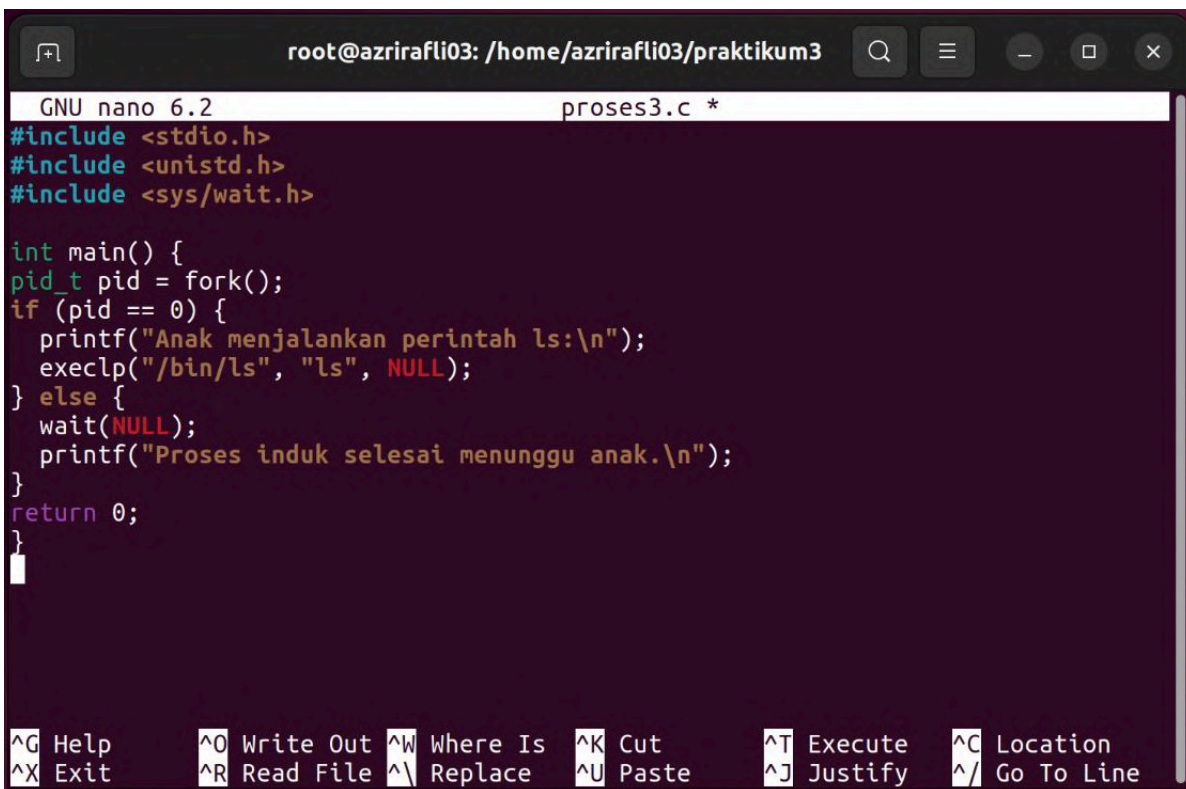
7. Analisis codingan dan hasil dari proses2.c sebagai berikut.
- Program mencetak sebelum **exec()**.
 - Lalu proses diganti dengan program **ls**.
 - Baris setelah **exec()** tidak dieksekusi jika exec berhasil.

3.3.3 Wait

Wait adalah system call yang digunakan oleh parent process untuk menunggu hingga child process selesai dieksekusi. Fungsi ini memastikan sinkronisasi proses serta mencegah terjadinya zombie process.

Berikut adalah langkah-langkah praktikum:

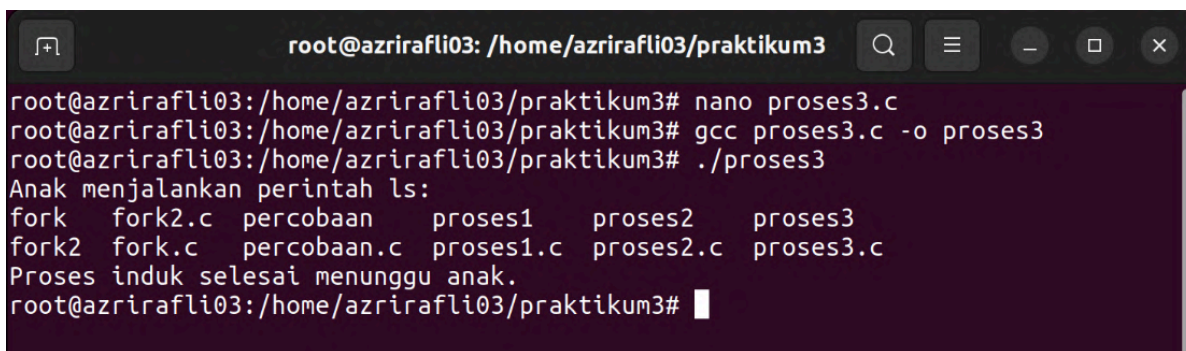
1. Buat proses untuk percobaan **wait()** dengan perintah **nano proses3.c**.
2. Isi codingan **proses3.c** sesuai gambar di bawah ini. Pastikan codingan sesuai dengan di gambar. Perhatikan simbol, besar dan kecilnya huruf.



```
GNU nano 6.2      proses3.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Anak menjalankan perintah ls:\n");
        execlp("/bin/ls", "ls", NULL);
    } else {
        wait(NULL);
        printf("Proses induk selesai menunggu anak.\n");
    }
    return 0;
}
```

3. Simpan sebagai proses3.c dengan menekan CTRL + X, lalu tekan y, dan lalu Enter.
4. Lakukan kompilasi file proses3.c menjadi proses3. Perintahnya pada bagian b).
 - a. Format perintah : **gcc nama_file.c -o nama_file**
 - b. **gcc proses3.c -o proses3**
5. Jalankan codingan atau file proses3.c dengan mengetik perintah **./proses3**
6. Outputnya akan seperti pada gambar di bawah ini.



```
root@azirafli03: /home/azirafli03/praktikum3
root@azirafli03:/home/azirafli03/praktikum3# nano proses3.c
root@azirafli03:/home/azirafli03/praktikum3# gcc proses3.c -o proses3
root@azirafli03:/home/azirafli03/praktikum3# ./proses3
Anak menjalankan perintah ls:
fork  fork2.c  percobaan  proses1  proses2  proses3
fork2  fork.c   percobaan.c  proses1.c  proses2.c  proses3.c
Proses induk selesai menunggu anak.
root@azirafli03:/home/azirafli03/praktikum3#
```

7. Analisis codingan dan hasil dari proses3.c sebagai berikut.

- a. Proses anak menjalankan perintah ls.
- b. Proses induk menunggu anak selesai menggunakan wait().
- c. Setelah anak selesai, induk melanjutkan.

3.3.4 Sleep()

sleep() pada sistem operasi berguna untuk memberikan jeda waktu setiap proses tertentu. Berikut adalah Langkah-langkah praktikum yang bisa dijalankan.

1. Buat proses untuk percobaan **sleep()** dengan perintah **nano sleep.c**.
2. Isi codingan **sleep.c** sesuai gambar di bawah ini. Pastikan codingan sesuai dengan di gambar. Perhatikan simbol, besar dan kecilnya huruf.



```

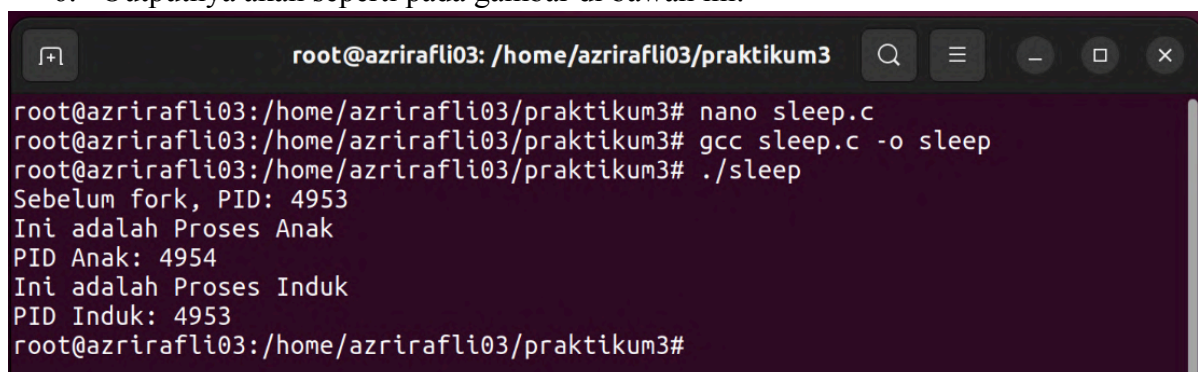
GNU nano 6.2
sleep.c *
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    pid_t pid, mypid, myppid;
    pid = getpid();
    printf("Sebelum fork, PID: %d\n", pid);
    pid = fork();

    if(pid < 0){
        perror("fork() Gagal\n");
        return 1;
    }

    // Proses Anak
    if(pid == 0) {
        printf("Ini adalah Proses Anak\n");
        pid = getpid();
        printf("PID Anak: %d\n", pid);
    } else { // Proses Induk
        sleep(2);
        printf("Ini adalah Proses Induk\n");
        pid = getpid();
        printf("PID Induk: %d\n", pid);
    }
    return 0;
}
  
```

3. Simpan sebagai sleep.c dengan menekan CTRL + X, lalu tekan y, dan lalu Enter.
4. Lakukan kompilasi file sleep.c menjadi sleep. Perintahnya pada bagian b).
 - a. Format perintah : **gcc nama_file.c -o nama_file**
 - b. **gcc sleep.c -o sleep**
5. Jalankan codingan atau file **sleep.c** dengan mengetik perintah **./sleep**
6. Outputnya akan seperti pada gambar di bawah ini.



```

root@azirafli03: /home/azirafli03/praktikum3
root@azirafli03:/home/azirafli03/praktikum3# nano sleep.c
root@azirafli03:/home/azirafli03/praktikum3# gcc sleep.c -o sleep
root@azirafli03:/home/azirafli03/praktikum3# ./sleep
Sebelum fork, PID: 4953
Ini adalah Proses Anak
PID Anak: 4954
Ini adalah Proses Induk
PID Induk: 4953
root@azirafli03:/home/azirafli03/praktikum3#
  
```

7. Analisis codingan dan hasil dari sleep.c sebagai berikut.
 - a. **Sleep()** akan melakukan jeda waktu terhadap proses tertentu.

- b. Terlihat bahwa perintah dari Proses Anak dijalankan lebih dahulu karena tidak terdapat perintah **sleep()**. Berbeda dengan Proses Induk yang terdapat jeda karena menggunakan fungsi **sleep()**.
- c. Durasi jeda tergantung inputan **sleep()**. Formatnya, yaitu **sleep(durasi)**.
- d. Misalkan **sleep(2)** yang artinya terdapat jeda selama 2 detik.

3.3.5 Melihat Proses di Sistem

Berikut adalah langkah-langkah untuk melihat proses yang berjalan di sistem:

1. Jalankan perintah **ps aux | head -n 10** pada terminal. Amati gambar di bawah ini.

```

root@azrirafli03: /home/azrirafli03/praktikum3
root@azrirafli03: /home/azrirafli03/praktikum3# nano sleep.c
root@azrirafli03: /home/azrirafli03/praktikum3# gcc sleep.c -o sleep
root@azrirafli03: /home/azrirafli03/praktikum3# ./sleep
Sebelum fork, PID: 4953
Ini adalah Proses Anak
PID Anak: 4954
Ini adalah Proses Induk
PID Induk: 4953
root@azrirafli03: /home/azrirafli03/praktikum3# ps aux | head -n 10
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2 166736 11516 ?        Ss   Des09    0:01 /sbin/init splash
root           2  0.0  0.0      0     0 ?        S    Des09    0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    Des09    0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   Des09    0:00 [kworker/R-rcu_g]
root           5  0.0  0.0      0     0 ?        I<   Des09    0:00 [kworker/R-rcu_p]
root           6  0.0  0.0      0     0 ?        I<   Des09    0:00 [kworker/R-slab_]
root           7  0.0  0.0      0     0 ?        I<   Des09    0:00 [kworker/R-netns]
root          10  0.0  0.0      0     0 ?        I<   Des09    0:00 [kworker/0:0H-events_highpri]
root          11  0.0  0.0      0     0 ?        I    Des09    0:00 [kworker/u6:0-ext4-rsv-conversion]
root@azrirafli03: /home/azrirafli03/praktikum3#

```

2. Selain perintah no.1, bisa juga menggunakan perintah **top** di bawah ini. Tekan tombol **CTRL + C** atau huruf **q** pada keyboard untuk keluar dari tampilan ini.

```

root@azrirafli03: /home/azrirafli03/praktikum3
root          6  0.0  0.0      0     0 ?        I<   Des09    0:00 [kworker/R-slab_]
root          7  0.0  0.0      0     0 ?        I<   Des09    0:00 [kworker/R-netns]
root         10  0.0  0.0      0     0 ?        I<   Des09    0:00 [kworker/0:0H-events_highpri]
root         11  0.0  0.0      0     0 ?        I    Des09    0:00 [kworker/u6:0-ext4-rsv-conversion]
root@azrirafli03: /home/azrirafli03/praktikum3# top

top - 02:34:43 up 3:45, 2 users, load average: 0.02, 0.04, 0.01
Tasks: 204 total, 1 running, 203 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.1 sy,  0.0 ni, 98.3 id,  0.0 wa,  0.0 hi,  1.6 si,  0.0 st
MiB Mem : 4251.0 total, 1132.9 free, 1018.9 used, 2099.2 buff/cache
MiB Swap: 1428.0 total, 1428.0 free,  0.0 used. 2929.5 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
 1402 azriraf+  20   0 5014652 553292 154452 S   2.0  12.7   2:34.26 gnome-shell
 2653 azriraf+  20   0 570440 60668 47012 S   0.3   1.4   0:10.21 gnome-terminal-
 4863 root       20   0      0      0      0 I   0.3   0.0   0:00.41 kworker/u9:2-events_unbound
 5037 root       20   0 13084  4096  3328 R   0.3   0.1   0:00.12 top
    1 root       20   0 166736 11516 8188 S   0.0   0.3   0:01.18 systemd
    2 root       20   0      0      0      0 S   0.0   0.0   0:00.02 kthreadd
    3 root       20   0      0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root       0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
    5 root       0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
    6 root       0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-slab_
    7 root       0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
   10 root       0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri

```

3. Analisis:
 - a. Amati kolom **PID, USER, %CPU, STAT, COMMAND**.
 - b. Identifikasi proses yang sedang berjalan.

MODUL 4 : Thread & Sinkronisasi

Kegiatan Belajar	Waktu (Jam) Belajar Mhs	Waktu (Jam) Dosen	Waktu (Jam) Asisten/Laboran
Tutorial / Pre- Test	0.5 Jam	Tidak hadir	Hadir
Persiapan	0.5 Jam	Tidak hadir	Hadir
Pelaksanaan Praktikum	4 Jam	Tidak hadir / Hadir	Hadir
Penyusunan Laporan	1.5 Jam	Tidak hadir	Hadir
Presentasi	0.5 Jam	Hadir	Hadir
Total Waktu (Menit)	7 Jam	0.5 Jam	6.5 Jam

4.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep dasar thread dan perbedaannya dengan proses.
2. Mempraktekkan pembuatan dan manipulasi thread.
3. Memahami pentingnya sinkronisasi thread.
4. Mengimplementasikan mekanisme sinkronisasi seperti mutex locks, semaphores, dan monitors.

4.2 Alat & Bahan

Alat & bahan yang digunakan adalah hardware perangkat PC beserta kelengkapannya berjumlah 40 PC dengan software aplikasi yang diinginkan dalam pembuatan program

1. Sistem operasi Linux Ubuntu (minimal versi 20.04).
2. Kompilator GCC untuk C atau interpreter Python.
3. Editor teks atau IDE seperti Visual Studio Code, Vim, Nano, atau PyCharm.
4. Terminal Linux untuk menjalankan perintah.

4.3 Dasar Teori

4.3.1 Thread

Thread adalah unit eksekusi terkecil dalam sebuah proses. Setiap proses minimal memiliki satu thread utama (main thread). Keunggulan thread:

- a. Thread lebih ringan daripada proses.
- b. Mendukung eksekusi paralel untuk mempercepat kinerja aplikasi.

4.3.2 Sinkronisasi

Ketika beberapa thread berbagi sumber daya yang sama, masalah race condition dapat muncul. Mekanisme sinkronisasi digunakan untuk menghindari konflik, seperti:

- Mutex Locks: Menjamin hanya satu thread dapat mengakses sumber daya secara bersamaan.
- Semaphores: Digunakan untuk mengontrol akses ke sumber daya terbatas.
- Monitors: Abstraksi tingkat tinggi untuk sinkronisasi.

4.3.3 Pengimplementasian Threads

Praktik 1: Membuat Thread

Kode Program

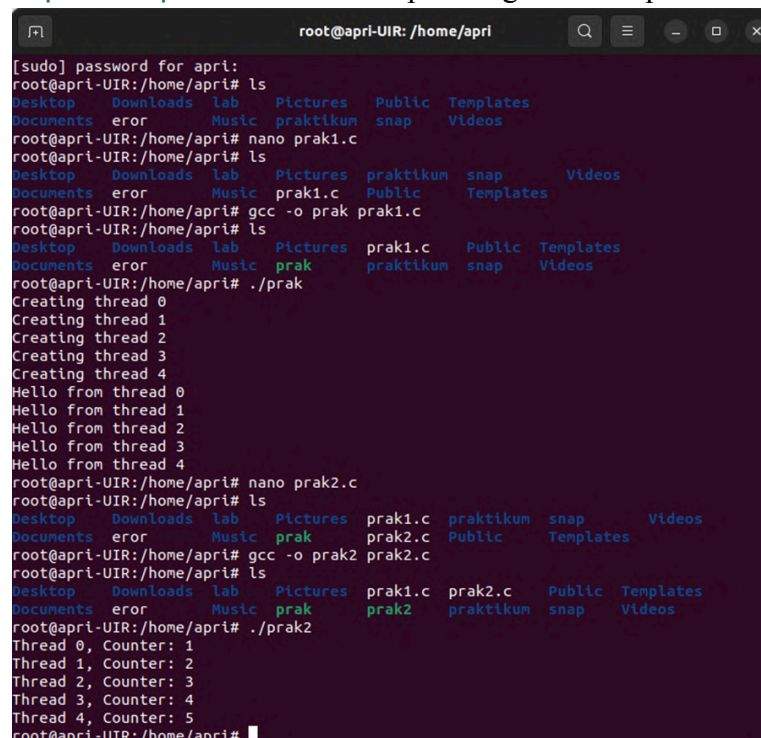
```
#include <stdio.h>
#include <pthread.h>

void* print_message(void* thread_id) {
    printf("Hello from thread %ld\n", (long) thread_id);
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[5];
    for (long i = 0; i < 5; i++) {
        printf("Creating thread %ld\n", i);
        pthread_create(&threads[i], NULL, print_message, (void*) i);
    }

    for (int i = 0; i < 5; i++) {
        pthread_join(threads[i], NULL);
    }
    return 0;
}
```

Ketik kode tersebut kemudian simpan dengan nama `prak1.c` kemudian jalankan dengan perintah `gcc -c prak1 prak1.c` lihat output dengan ketik `./prak1`



```
root@apri-UIR: /home/apri
[sudo] password for apri:
root@apri-UIR:/home/apri# ls
Desktop  Downloads  lab  Pictures  Public  Templates
Documents  error  Music  praktikum  snap  Videos
root@apri-UIR:/home/apri# nano prak1.c
root@apri-UIR:/home/apri# ls
Desktop  Downloads  lab  Pictures  praktikum  snap  Videos
Documents  error  Music  prak1.c  Public  Templates
root@apri-UIR:/home/apri# gcc -o prak prak1.c
root@apri-UIR:/home/apri# ls
Desktop  Downloads  lab  Pictures  prak1.c  Public  Templates
Documents  error  Music  prak  praktikum  snap  Videos
root@apri-UIR:/home/apri# ./prak
Creating thread 0
Creating thread 1
Creating thread 2
Creating thread 3
Creating thread 4
Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3
Hello from thread 4
root@apri-UIR:/home/apri# nano prak2.c
root@apri-UIR:/home/apri# ls
Desktop  Downloads  lab  Pictures  prak1.c  praktikum  snap  Videos
Documents  error  Music  prak  prak2.c  Public  Templates
root@apri-UIR:/home/apri# gcc -o prak2 prak2.c
root@apri-UIR:/home/apri# ls
Desktop  Downloads  lab  Pictures  prak1.c  prak2.c  Public  Templates
Documents  error  Music  prak  prak2  praktikum  snap  Videos
root@apri-UIR:/home/apri# ./prak2
Thread 0, Counter: 1
Thread 1, Counter: 2
Thread 2, Counter: 3
Thread 3, Counter: 4
Thread 4, Counter: 5
root@apri-UIR:/home/apri#
```

Praktik 2: Sinkronisasi dengan Mutex

Kode Program

```
#include <stdio.h>
#include <pthread.h>
int counter = 0;
pthread_mutex_t lock;
void* increment_counter(void* thread_id) {
    pthread_mutex_lock(&lock);
    counter++;
    printf("Thread %ld, Counter: %d\n", (long)thread_id, counter);
    pthread_mutex_unlock(&lock);
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[5];
    pthread_mutex_init(&lock, NULL);

    for (long i = 0; i < 5; i++) {
        pthread_create(&threads[i], NULL, increment_counter, (void*)i);
    }

    for (int i = 0; i < 5; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);
    return 0;
}
```

Praktik 3: Sinkronisasi dengan Semaphore

Kode Program

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
sem_t semaphore;
void* critical_section(void* thread_id) {
    sem_wait(&semaphore);
    printf("Thread %ld entering critical section.\n", (long) thread_id);
    sleep(1);
    printf("Thread %ld leaving critical section.\n", (long) thread_id);
    sem_post(&semaphore);
    pthread_exit(NULL);
}
```

```

int main() {
    pthread_t threads[5];
    sem_init(&semaphore, 0, 2);
    for (long i = 0; i < 5; i++) {
        pthread_create(&threads[i], NULL, critical_section, (void*) i);
    }
    for (int i = 0; i < 5; i++) {
        pthread_join(threads[i], NULL);
    }
    sem_destroy(&semaphore);
    return 0;
}

```

Praktikum 4: Amdahl Law

ketik perintah :

```

#include <stdio.h>

// Fungsi untuk menghitung speedup berdasarkan hukum Amdahl

double hitung_speedup(double P, int N) {
    return 1 / ((1 - P) + (P / N));
}

int main() {
    double P; // Proporsi program yang bisa diparalelkan
    int N;     // Jumlah core/prosesor
    double speedup;

    // Meminta input dari pengguna
    printf("Masukkan proporsi bagian yang dapat diparalelkan (0 - 1): ");
    scanf("%lf", &P);

    printf("Masukkan jumlah core: ");
    scanf("%d", &N);

    // Menghitung percepatan

```

```

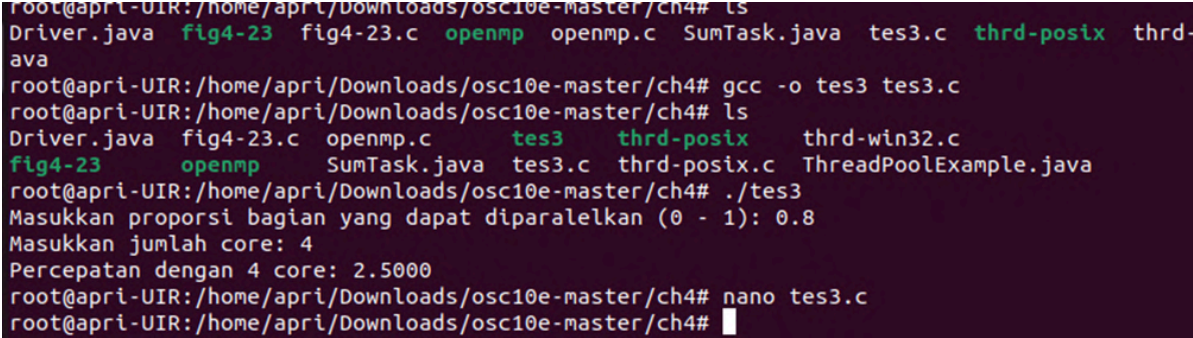
speedup = hitung_speedup(P, N);

// Menampilkan hasil percepatan

printf("Percepatan dengan %d core: %.4f\n", N, speedup);

return 0;
}

```



```

root@apri-UIR:/home/apri/Downloads/osc10e-master/ch4# ls
Driver.java  fig4-23  fig4-23.c  openmp  openmp.c  SumTask.java  tes3.c  thrd-posix  thrd-
ava
root@apri-UIR:/home/apri/Downloads/osc10e-master/ch4# gcc -o tes3 tes3.c
root@apri-UIR:/home/apri/Downloads/osc10e-master/ch4# ls
Driver.java  fig4-23.c  openmp.c  tes3  thrd-posix  thrd-win32.c
fig4-23  openmp  SumTask.java  tes3.c  thrd-posix.c  ThreadPoolExample.java
root@apri-UIR:/home/apri/Downloads/osc10e-master/ch4# ./tes3
Masukkan proporsi bagian yang dapat diparalelkan (0 - 1): 0.8
Masukkan jumlah core: 4
Percepatan dengan 4 core: 2.5000
root@apri-UIR:/home/apri/Downloads/osc10e-master/ch4# nano tes3.c
root@apri-UIR:/home/apri/Downloads/osc10e-master/ch4#

```

4.4 Latihan

Buat dalam bentuk Laporan dengan format PDF, Selesaikan pernyataan dibawah:

1. Jelaskan secara ringkas dan jelas konsep-konsep berikut:
 - Thread
 - Critical Section
 - Race Condition (beserta contoh)
 - Mutex
 - Semaphore
 - Hukum Amdahl (beserta rumus dan contoh hitung manual)
2. Implementasikan Proses pada Materi pada Ubuntu masing-masing lalu:
 - Jelaskan tujuan program yang dibuat
 - Screenshot code program dan command linanya
 - Jelaskan fungsi utama program dan peran setiap bagian function (method)
 - Tampilkan hasil compile program
3. Kesimpulan mengenai:
 - Pentingnya thread dan sinkronisasi
 - Dampak race condition terhadap program
 - Hubungan antara jumlah thread dan performa berdasarkan Hukum Amdahl

MODUL 5 : Penjadwalan

Kegiatan Belajar	Waktu (Jam) Belajar Mhs	Waktu (Jam) Dosen	Waktu (Jam) Asisten/Laboran
Tutorial / Pre- Test	0.5 Jam	Tidak hadir	Hadir
Persiapan	0.5 Jam	Tidak hadir	Hadir
Pelaksanaan Praktikum	4 Jam	Tidak hadir / Hadir	Hadir
Penyusunan Laporan	1.5 Jam	Tidak hadir	Hadir
Presentasi	0.5 Jam	Hadir	Hadir
Total Waktu (Menit)	7 Jam	0.5 Jam	6.5 Jam

5.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep dasar algoritma penjadwalan proses dalam sistem operasi.
2. Mengimplementasikan algoritma First-Come-First-Served (FCFS), Shortest Job First (SJF), Priority Scheduling, dan Round Robin (RR).
3. Membandingkan performa algoritma berdasarkan waktu tunggu rata-rata (average waiting time) dan waktu turn-around rata-rata (average turnaround time).

5.2 Alat & Bahan

Alat & bahan yang digunakan adalah hardware perangkat PC beserta kelengkapannya berjumlah 40 PC dengan software aplikasi yang diinginkan dalam pembuatan program

5. Sistem operasi Linux Ubuntu (minimal versi 20.04).
6. Kompilator GCC untuk C atau interpreter Python.
7. Editor teks atau IDE seperti Visual Studio Code, Vim, Nano, atau PyCharm.
8. Terminal Linux untuk menjalankan perintah.

5.3 Dasar Teori

5.3.1 Konsep Proses dalam Sistem Operasi

Proses adalah sebuah program yang sedang dieksekusi oleh sistem komputer. Setiap proses membutuhkan sumber daya sistem seperti CPU, memori, dan perangkat input/output. Dalam sistem operasi multitasking, terdapat banyak proses yang berjalan secara bersamaan sehingga diperlukan mekanisme pengaturan agar CPU dapat digunakan secara efisien.

5.3.2 Penjadwalan Proses

Penjadwalan proses adalah mekanisme yang digunakan oleh sistem operasi untuk menentukan urutan eksekusi proses yang akan dijalankan oleh CPU. Tujuan utama penjadwalan proses adalah untuk meningkatkan penggunaan CPU, mempercepat waktu respons, dan memberikan pembagian penggunaan setiap proses yang berjalan.

5.3.3 Kriteria Penjadwalan

Beberapa kriteria yang digunakan dalam mengevaluasi algoritma penjadwalan antara lain:

1. Waiting Time – waktu tunggu proses di antrian siap (ready queue).
2. Turnaround Time – total waktu dari proses mulai masuk hingga selesai dieksekusi.
3. Response Time – waktu dari proses mulai masuk hingga mendapatkan CPU pertama kali.
4. Throughput – jumlah proses yang dapat diselesaikan dalam satu satuan waktu.

5. CPU Utilization – tingkat pemanfaatan CPU selama sistem berjalan.

5.3.4 Jenis-Jenis Algoritma Penjadwalan

1. First Come First Served (FCFS)

FCFS merupakan algoritma penjadwalan paling sederhana, di mana proses dieksekusi berdasarkan urutan kedatangan. Proses yang datang lebih awal akan dijalankan terlebih dahulu.

2. Shortest Job First (SJF)

SJF mengeksekusi proses dengan waktu eksekusi (burst time) paling pendek terlebih dahulu. Algoritma ini dapat meminimalkan waktu tunggu rata-rata.

3. Priority Scheduling

Pada algoritma ini, setiap proses memiliki tingkat prioritas. Proses dengan prioritas tertinggi akan dieksekusi terlebih dahulu.

4. Round Robin (RR)

Round Robin membagi waktu CPU menjadi potongan waktu tetap yang disebut time quantum. Setiap proses akan mendapatkan giliran CPU secara bergantian.

5.3.5 Peran Penjadwalan dalam Sistem Operasi

Penjadwalan proses berperan penting dalam menjaga kinerja sistem operasi agar tetap responsif dan stabil. Algoritma penjadwalan yang tepat dapat meningkatkan efisiensi sistem serta kenyamanan pengguna dalam menjalankan aplikasi.

5.4 Latihan

Buatlah latihan sebagai berikut dan ikuti langkah langkah sebagai berikut

Step 1:

```
randy@randy:~$ sudo su
[sudo] password for randy:
root@randy:/home/randymkdir p6
root@randy:/home/randymkdir cd p6
root@randy:/home/randyp6# nano fcsf.c
```

Step 2:

```

GNU nano 6.2                                fcsf.c *
#include <stdio.h>

void findWaitingTime(int processes[], int n, int bt[], int wt[]) {
    wt[0] = 0;
    for (int i = 1; i < n; i++) {
        wt[i] = bt[i - 1] + wt[i - 1];
    }
}

void findTurnAroundTime(int processes[], int n, int bt[], int wt[], int tat[]) {
    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
    }
}

void findAvgTime(int processes[], int n, int bt[]) {
    int wt[n], tat[n];
    int total_wt = 0, total_tat = 0;

    findWaitingTime(processes, n, bt, wt);
    findTurnAroundTime(processes, n, bt, wt, tat);

    printf("Processes\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (int i = 0; i < n; i++) {
        total_wt += wt[i];
        total_tat += tat[i];
        printf("%d\t%d\t%d\t%d\n",
            processes[i], bt[i], wt[i], tat[i]);
    }

    printf("Average Waiting Time: %.2f\n", (float)total_wt / n);
    printf("Average Turnaround Time: %.2f\n", (float)total_tat / n);
}

```

Step 3:

```

int main() {
    int processes[] = {1, 2, 3};
    int burst_time[] = {10, 5, 8};
    int n = sizeof(processes) / sizeof(processes[0]);

    findAvgTime(processes, n, burst_time);
    return 0;
}

```

Step 4:

```

root@randy:/home/randyp6# gcc -o fcsf fcsf.c
root@randy:/home/randyp6# ./fcsf
Processes      Burst Time      Waiting Time      Turnaround Time
1              10              0                10
2              5              10              15
3              8              15              23
Average Waiting Time: 8.33
Average Turnaround Time: 16.00

```

Step 5:

```

GNU nano 6.2                                sjf.c *
#include <stdio.h>

void sortProcesses(int processes[], int n, int bt[]) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (bt[j] > bt[j + 1]) {
                int temp = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = temp;

                temp = processes[j];
                processes[j] = processes[j + 1];
                processes[j + 1] = temp;
            }
        }
    }
}

void findAvgTime(int processes[], int n, int bt[]) {
    int wt[n], tat[n];
    int total_wt = 0, total_tat = 0;

    sortProcesses(processes, n, bt);

    wt[0] = 0;
    for (int i = 1; i < n; i++) {
        wt[i] = bt[i - 1] + wt[i - 1];
    }

    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("Processes\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\n",
            processes[i], bt[i], wt[i], tat[i]);
    }

    printf("Average Waiting Time: %.2f\n", (float)total_wt / n);
    printf("Average Turnaround Time: %.2f\n", (float)total_tat / n);
}

int main() {
    int processes[] = {1, 2, 3};
    int burst_time[] = {6, 8, 7};
    int n = sizeof(processes) / sizeof(processes[0]);

    findAvgTime(processes, n, burst_time);
    return 0;
}

```

Step 6:

```

root@randy:/home/randy/p6# gcc -o sjf sjf.c
root@randy:/home/randy/p6# ./sjf
Processes      Burst Time      Waiting Time      Turnaround Time
1              6              0              6
3              7              6              13
2              8              13             21
Average Waiting Time: 6.33
Average Turnaround Time: 13.33
root@randy:/home/randy/p6#

```

Step 7:

```

GNU nano 6.2 priority
#include <stdio.h>

void sortPriority(int processes[], int n, int bt[], int priority[]) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (priority[j] < priority[j + 1]) {
                int temp = priority[j];
                priority[j] = priority[j + 1];
                priority[j + 1] = temp;

                temp = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = temp;

                temp = processes[j];
                processes[j] = processes[j + 1];
                processes[j + 1] = temp;
            }
        }
    }
}

void findAvgTime(int processes[], int n, int bt[], int priority[]) {
    int wt[n], tat[n];
    int total_wt = 0, total_tat = 0;

    sortPriority(processes, n, bt, priority);

    wt[0] = 0;
    for (int i = 1; i < n; i++) {
        wt[i] = bt[i - 1] + wt[i - 1];
    }

    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("Processes\tPriority\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\n",
            processes[i], priority[i], bt[i], wt[i], tat[i]);
    }

    printf("Average Waiting Time: %.2f\n", (float)total_wt / n);
    printf("Average Turnaround Time: %.2f\n", (float)total_tat / n);
}

int main() {
    int processes[] = {1, 2, 3};
    int burst_time[] = {10, 5, 8};
    int priority[] = {1, 3, 2};
    int n = sizeof(processes) / sizeof(processes[0]);

    findAvgTime(processes, n, burst_time, priority);
    return 0;
}

```

Step 8:

```

root@randy:/home/randy/p6# gcc -o round_robin round_robin.c
root@randy:/home/randy/p6# ./round_robin
Processes      Burst Time      Waiting Time      Turnaround Time
1              10             13              23
2              5              10              15
3              8              13              21
Average Waiting Time: 12.00
Average Turnaround Time: 19.67

```

MODUL 6 : Deadlock

Kegiatan Belajar	Waktu (Jam) Belajar Mhs	Waktu (Jam) Dosen	Waktu (Jam) Asisten/Laboran
Tutorial / Pre- Test	0.5 Jam	Tidak hadir	Hadir
Persiapan	0.5 Jam	Tidak hadir	Hadir
Pelaksanaan Praktikum	4 Jam	Tidak hadir / Hadir	Hadir
Penyusunan Laporan	1.5 Jam	Tidak hadir	Hadir
Presentasi	0.5 Jam	Hadir	Hadir
Total Waktu (Menit)	7 Jam	0.5 Jam	6.5 Jam

6.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep dasar deadlock dalam sistem operasi.
2. Mengidentifikasi kondisi-kondisi yang menyebabkan deadlock.
3. Mengimplementasikan dan menguji simulasi deadlock dalam aplikasi Java.
4. Menganalisis cara-cara untuk mendeteksi dan mengatasi deadlock dalam aplikasi multithreading.

6.2 Alat & Bahan

Alat & bahan yang digunakan adalah hardware perangkat PC beserta kelengkapannya berjumlah 40 PC dengan software aplikasi yang diinginkan dalam pembuatan program

6.3 Dasar Teori

6.3.1. Penjelasan Konsep Deadlock

Deadlock terjadi ketika dua atau lebih proses dalam sistem saling menunggu satu sama lain untuk melepaskan sumber daya yang dibutuhkan, dan tidak ada satu pun dari mereka yang bisa melanjutkan eksekusinya. Deadlock terjadi jika semua empat kondisi berikut ini terpenuhi:

1. Mutual Exclusion: Setidaknya satu sumber daya harus dalam keadaan terkunci (tidak dapat digunakan oleh lebih dari satu proses).
2. Hold and Wait: Proses yang sudah memegang satu sumber daya dapat menunggu untuk mendapatkan sumber daya lainnya.
3. No Preemption: Sumber daya tidak dapat dipaksa untuk dilepaskan dari proses yang memegangnya.
4. Circular Wait: Ada sebuah rangkaian proses, di mana setiap proses menunggu sumber daya yang sedang dikunci oleh proses lainnya dalam rangkaian tersebut.

6.3.2 Mengimplementasikan Deadlock dalam Java

1. Langkah 1: Membuat file DeadlockExample.java

```
public class DeadlockExample {  
    private static final Object lock1 = new Object();  
    private static final Object lock2 = new Object();  
    public static void main(String[] args) {  
        // Membuat dua thread yang akan menyebabkan deadlock  
        Thread thread1 = new Thread(new Task1());  
        Thread thread2 = new Thread(new Task2());
```

```

        thread1.start();
        thread2.start();
    }

    // Task pertama yang mencoba mengunci lock1 kemudian lock2
    static class Task1 implements Runnable {
        @Override
        public void run() {
            synchronized (lock1) {
                System.out.println("Task1: Memegang lock1");
                try { Thread.sleep(100); } catch (InterruptedException e) {}

                synchronized (lock2) {
                    System.out.println("Task1: Memegang lock2");
                }
            }
        }
    }

    // Task kedua yang mencoba mengunci lock2 kemudian lock1
    static class Task2 implements Runnable {
        @Override
        public void run() {
            synchronized (lock2) {
                System.out.println("Task2: Memegang lock2");
                try { Thread.sleep(100); } catch (InterruptedException e) {}

                synchronized (lock1) {
                    System.out.println("Task2: Memegang lock1");
                }
            }
        }
    }
}

```

2. Langkah 2: Penjelasan Kode:

DeadlockExample adalah kelas utama yang berisi dua objek lock, lock1 dan lock2, yang akan digunakan untuk mensimulasikan deadlock. Dua kelas Task1 dan Task2 masing-masing mencoba mengunci dua objek lock dalam urutan yang berbeda, yang mengarah pada kondisi Circular Wait, yaitu:

Task1 mengunci lock1 dan menunggu lock2.
Task2 mengunci lock2 dan menunggu lock1.

Karena Task1 dan Task2 saling menunggu satu sama lain untuk melepaskan lock yang mereka butuhkan, deadlock terjadi.

3. Menjalankan Deadlock

- a. Langkah 1: Kompilasi dan Jalankan Program
 - i. Buka terminal atau command prompt.
 - ii. Arahkan ke direktori tempat file DeadlockExample.java disimpan.

- iii. **Kompilasi file Java:** `javac DeadlockExample.java`
- iv. **Jalankan program:** `java DeadlockExample`

b. Langkah 2 : Analisis Output

Setelah program dijalankan, kamu akan melihat bahwa thread pertama (Task1) akan mengunci lock1, dan thread kedua (Task2) akan mengunci lock2. Kemudian, Task1 mencoba untuk mengunci lock2 yang sudah terkunci oleh Task2, dan sebaliknya, Task2 mencoba untuk mengunci lock1 yang sudah terkunci oleh Task1. Kedua thread tersebut saling menunggu tanpa dapat melanjutkan eksekusinya, yang menyebabkan deadlock. Output yang akan terlihat di terminal adalah:

Task1: Memegang lock1
Task2: Memegang lock2

Setelah itu, kedua thread akan berhenti dan tidak ada yang dapat melanjutkan.

4. Menangani Deadlock

a. Langkah 1 : Strategi Pembatalan Deadlock :

- i. Menggunakan Timeout: Jika thread tidak bisa mendapatkan semua sumber daya yang dibutuhkan dalam waktu tertentu, thread dapat dibatalkan dan sumber daya dilepaskan.
- ii. Menggunakan Lock yang Urutannya Konsisten: Dengan memastikan bahwa semua thread mengunci sumber daya dalam urutan yang sama, kita bisa mencegah deadlock.
- iii. Menggunakan Lock dengan Kondisi: Kamu bisa menggunakan *ReentrantLock* dengan metode *tryLock()* untuk mencoba mendapatkan lock dalam batas waktu tertentu.

- b. Langkah 2: Implementasi Menggunakan Timeout (Alternatif untuk Menghindari Deadlock) Untuk menangani deadlock, kamu bisa menggunakan *tryLock* dari *ReentrantLock* untuk mencoba mengunci sumber daya dengan batas waktu.

Membuat file *DeadlockExampleWithTimeout.java*

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
public class DeadlockExampleWithTimeout {
    private static final Lock lock1 = new ReentrantLock();
    private static final Lock lock2 = new ReentrantLock();
    public static void main(String[] args) {
        // Membuat dua thread yang akan mencoba menghindari deadlock
        Thread thread1 = new Thread(new Task1());
        Thread thread2 = new Thread(new Task2());

        thread1.start();
        thread2.start();
    }
}
```

```

// Task pertama yang mencoba mengunci lock1 kemudian lock2 dengan
timeout
static class Task1 implements Runnable {
    @Override
    public void run() {
        try {
            if (lock1.tryLock() && lock2.tryLock()) {
                System.out.println("Task1: Memegang lock1 dan lock2");
                // Simulasikan kerja
                Thread.sleep(100);
            } else {
                System.out.println("Task1: Gagal memperoleh lock, mencoba
lagi");
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            lock1.unlock();
            lock2.unlock();
        }
    }
}

// Task kedua yang mencoba mengunci lock2 kemudian lock1 dengan
timeout
static class Task2 implements Runnable {
    @Override
    public void run() {
        try {
            if (lock2.tryLock() && lock1.tryLock()) {
                System.out.println("Task2: Memegang lock2 dan lock1");
                // Simulasikan kerja
                Thread.sleep(100);
            } else {
                System.out.println("Task2: Gagal memperoleh lock, mencoba
lagi");
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            lock2.unlock();
            lock1.unlock();
        }
    }
}

```

Jalankan program:

1. `javac DeadlockExampleTimeout.java`
2. `java DeadlockExampleTimeout`

6.4 Latihan

Buatlah folder dengan nama Deadlock dan file didalam nya dengan nama DeadlockExample.java dan DeadlockExample.java

```
Task1: Memegang lock1  
Task2: Memegang lock2
```

Buatlah folder dengan nama Deadlock dan file didalam nya dengan nama DeadlockExample.java dan DeadlockExampleWithTimeout.java

```
Task 1: Berhasil memegang lock1 & lock2  
Task 2: Lock2 sibuk, mencoba lagi...  
Task 1: Selesai.  
Task 2: Berhasil memegang lock2 & lock1  
Task 2: Selesai.
```

MODUL 7 : Main Memory

Kegiatan Belajar	Waktu (Jam) Belajar Mhs	Waktu (Jam) Dosen	Waktu (Jam) Asisten/Laboran
Tutorial / Pre- Test	0.5 Jam	Tidak hadir	Hadir
Persiapan	0.5 Jam	Tidak hadir	Hadir
Pelaksanaan Praktikum	4 Jam	Tidak hadir / Hadir	Hadir
Penyusunan Laporan	1.5 Jam	Tidak hadir	Hadir
Presentasi	0.5 Jam	Hadir	Hadir
Total Waktu (Menit)	7 Jam	0.5 Jam	6.5 Jam

7.1 Tujuan

1. Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:
2. Melakukan Kalkulasi Translasi Alamat: Mampu menghitung pembagian alamat memori ke dalam *page number* (nomor halaman) dan *offset* (posisi data) dengan standar ukuran 4 KB.
3. Mengoperasikan Simulasi Alokasi Memori: Dapat menjalankan prosedur pemberian ruang memori bagi proses-proses baru melalui program simulasi yang telah disediakan.
4. Menganalisis Perbandingan Strategi Alokasi: Mampu membedakan serta menguji efektivitas algoritma *First Fit* (F), *Best Fit* (B), dan *Worst Fit* (W) dalam menempatkan proses.
5. Mengelola Siklus Hidup Memori: Mahir dalam menangani prosedur *Request* (permintaan memori), *Release* (pelepasan proses), serta memantau kondisi RAM melalui fitur *Stat* (status).
6. Menerapkan Teknik Kompaksi: Memahami sekaligus mensimulasikan metode *Compaction* (perapatan memori) untuk menyatukan lubang-lubang memori yang terpecah akibat fragmentasi eksternal.

7.2 Alat & Bahan

Praktikum ini menggunakan sistem virtualisasi agar simulasi manajemen memori berjalan stabil tanpa mengganggu sistem operasi utama pada laptop. Berikut rincian alat dan bahan yang digunakan:

1. Laptop/PC: Perangkat keras utama untuk menjalankan seluruh sistem.
2. VirtualBox: Aplikasi penyedia lingkungan mesin virtual.
3. Ubuntu 22.04 LTS: Sistem operasi berbasis Linux yang menjadi platform kerja.
4. Terminal: Antarmuka untuk menginput perintah dan memantau status memori.
5. GCC: Alat untuk mengkompilasi kode program bahasa C menjadi file eksekusi.
6. Kode Sumber: Berkas program utama yang berisi logika algoritma alokasi memori (*First Fit*, *Best Fit*, dan *Worst Fit*).

7.3 Dasar Teori

7.3.1 Ruang Alamat Logika Dan Fisik

Table Alamat Logika (Logical Address) dan Alamat Fisik (Physical Address) :

Kriteria	Alamat Logika (Logical Address)	Alamat Fisik (Physical Address)
Pembangkit	Dihasilkan oleh CPU selama eksekusi program.	Dihasilkan oleh Memory Management Unit (MMU).
Ruang Alamat	Ruang Alamat Logika (Logical Address Space) adalah kumpulan semua alamat logika yang dihasilkan oleh CPU sebagai referensi ke suatu program.	Ruang Alamat Fisik (Physical Address Space) adalah kumpulan semua alamat fisik yang dipetakan ke alamat logika yang bersesuaian.
Akses Pengguna	Pengguna dapat melihat dan mengakses alamat logika suatu program.	Pengguna tidak pernah dapat melihat alamat fisik program.
Perubahan Saat Eksekusi	Dapat berubah selama eksekusi program (misalnya karena relokasi, <i>paging</i> , dll.).	Umumnya tetap setelah ditetapkan di memori.
Akses ke Alamat Lain	Pengguna dapat menggunakan alamat logika untuk mengakses alamat fisik.	Pengguna dapat mengakses alamat fisik secara tidak langsung, tetapi tidak secara langsung.
Nama Lain	Alamat Virtual (<i>Virtual Address</i>).	Alamat Riil (<i>Real Address</i>).

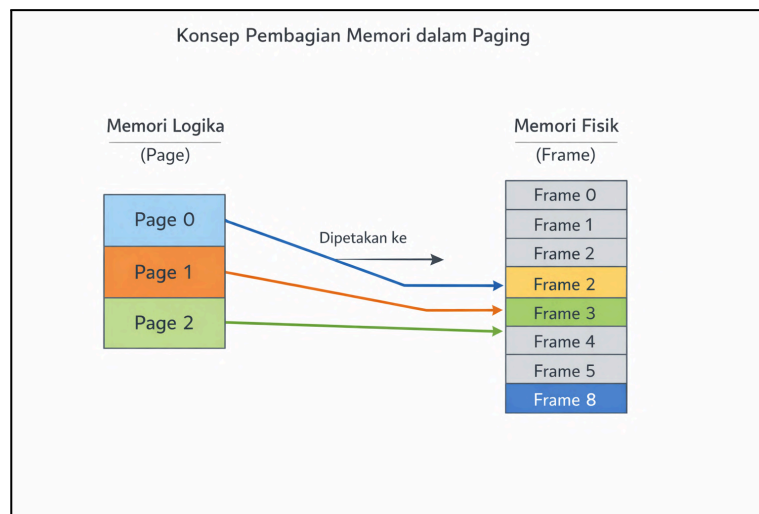
7.3.2 Mekanisme Paging

Paging merupakan skema manajemen memori yang memungkinkan ruang alamat fisik suatu proses menjadi tidak berurutan (*non-contiguous*). Teknik ini menghilangkan kebutuhan untuk menempatkan proses pada satu blok memori yang menyambung, sehingga penggunaan *RAM* menjadi lebih efisien. Dalam skema ini, memori fisik dipecah menjadi blok-blok berukuran tetap yang disebut *frame*, sementara memori logika (program) dipecah menjadi blok-blok dengan ukuran yang sama yang disebut *page* (halaman)

7.3.2.1 Konsep Pembagian Memori dalam Paging

mekanisme paging, sistem operasi membagi memori logika dan memori fisik ke dalam blok-blok berukuran tetap. Memori logika yang digunakan oleh program dibagi menjadi page (halaman), sedangkan memori fisik atau RAM dibagi menjadi frame. Pembagian ini bertujuan agar sistem operasi dapat menempatkan bagian-bagian program ke dalam memori fisik tanpa harus tersusun secara berurutan.

Setiap page memiliki ukuran yang sama dengan frame, sehingga satu page dapat ditempatkan tepat ke dalam satu frame. Dengan konsep ini, halaman-halaman suatu program dapat tersebar di berbagai lokasi memori fisik, namun tetap dapat diakses secara logis dan teratur. Pembagian memori dalam paging membantu meningkatkan efisiensi penggunaan memori serta mengurangi fragmentasi eksternal.



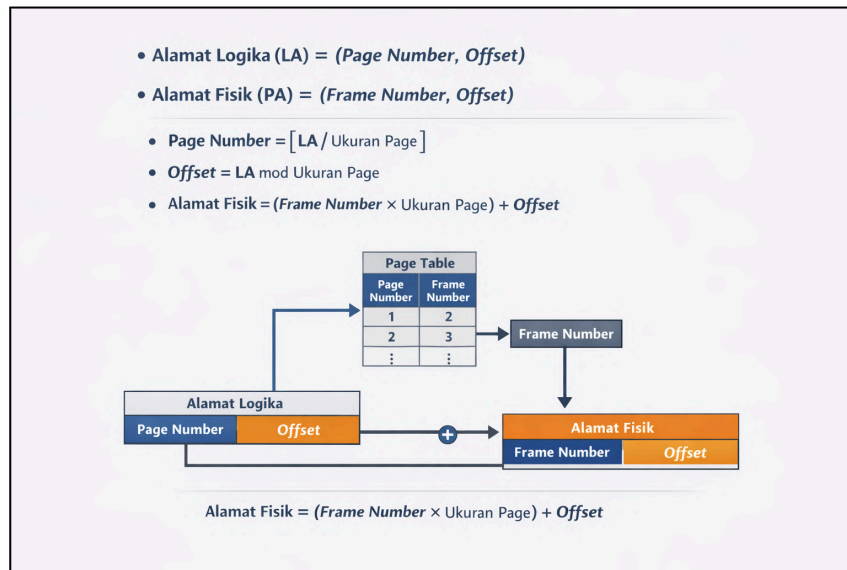
7.3.2.2 Ukuran Halaman Tetap (Fixed Page Size)

Ukuran Halaman Tetap (Fixed Page Size) adalah metode manajemen memori pada sistem operasi di mana memori utama dibagi menjadi sejumlah blok dengan ukuran yang sama, yang disebut *frame*, sementara ruang alamat proses dibagi menjadi bagian-bagian berukuran sama yang disebut *page*. Setiap page dari suatu proses dapat ditempatkan ke frame mana pun di memori utama tanpa harus berurutan. Pendekatan ini bertujuan untuk menyederhanakan proses alokasi dan pengelolaan memori, karena sistem tidak perlu mencari ruang memori dengan ukuran tertentu. Selain itu, metode ini mendukung penggunaan memori virtual dan memungkinkan proses *paging* antara memori utama dan media penyimpanan sekunder. Meskipun demikian, penggunaan ukuran halaman tetap dapat menyebabkan *fragmentasi internal*, yaitu kondisi dimana sebagian ruang dalam frame tidak terpakai apabila ukuran data lebih kecil dari ukuran halaman yang ditetapkan.

7.3.2.3 Struktur Alamat Logika pada Paging

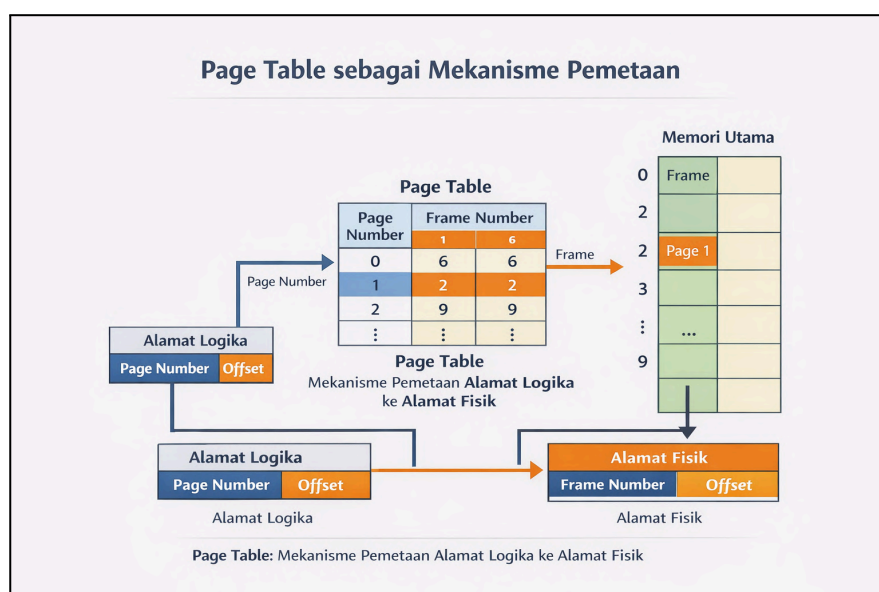
Struktur Alamat Logika pada Paging adalah mekanisme pengalamatan memori dalam sistem operasi yang menggunakan teknik paging, di mana alamat logika yang dihasilkan oleh CPU dibagi menjadi dua bagian, yaitu *page number* dan *offset*. *Page number* digunakan sebagai indeks untuk mengakses *page table*, yang berisi pemetaan antara halaman logika dan *frame* di memori utama. Sementara itu, *offset* menunjukkan posisi data di dalam suatu halaman dan tetap bernilai sama selama proses translasi alamat berlangsung. Setelah sistem memperoleh *frame number* dari *page table*, alamat fisik dibentuk dengan menggabungkan *frame number* dan *offset*,

sehingga memungkinkan proses untuk diakses meskipun halaman-halamannya tidak tersusun secara berurutan di memori utama. Struktur ini menjadi dasar dari implementasi memori virtual dan memungkinkan penggunaan memori yang lebih efisien serta pengelolaan proses yang fleksibel.



7.3.2.4 Page Table sebagai Mekanisme Pemetaan

Page Table sebagai Mekanisme Pemetaan adalah struktur data yang digunakan oleh sistem operasi untuk memetakan alamat logika ke alamat fisik pada sistem memori berbasis paging. Page table berisi daftar entri yang menghubungkan page number dari alamat logika dengan frame number di memori utama. Ketika CPU menghasilkan alamat logika, page number digunakan sebagai indeks untuk mengakses page table, sehingga sistem dapat mengetahui frame tempat halaman tersebut disimpan. Informasi ini kemudian digabungkan dengan offset untuk membentuk alamat fisik yang valid. Mekanisme ini memungkinkan halaman-halaman suatu proses ditempatkan secara tidak berurutan di memori utama, mendukung implementasi memori virtual, serta meningkatkan efisiensi pemanfaatan memori. Selain pemetaan, page table juga dapat menyimpan informasi tambahan seperti bit valid, bit proteksi, dan status akses halaman.

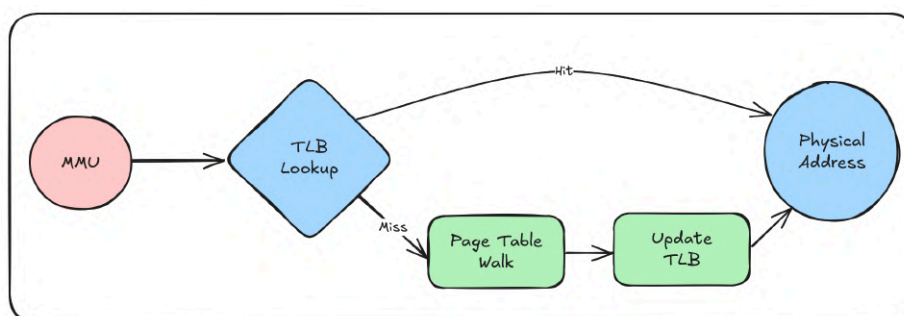


7.3.2.5 Peran Memory Management Unit (MMU) dalam Paging

Memory Management Unit (MMU) adalah komponen perangkat keras yang berfungsi melakukan penerjemahan alamat logika menjadi alamat fisik pada sistem memori yang menerapkan teknik paging. Dalam mekanisme ini, MMU menerima alamat logika dari CPU, kemudian memisahnya menjadi *page number* dan *offset*. *Page number* digunakan untuk mengakses *page table* guna memperoleh *frame number* yang sesuai di memori utama, sedangkan *offset* tetap dipertahankan nilainya. Setelah proses pemetaan selesai, MMU membentuk alamat fisik dengan menggabungkan *frame number* dan *offset*, lalu meneruskannya ke memori utama. Selain melakukan translasi alamat, MMU juga berperan dalam pengawasan proteksi memori, seperti pemeriksaan hak akses halaman dan pendeteksian kesalahan akses (*page fault*). Dengan demikian, MMU memungkinkan implementasi memori virtual berjalan secara efisien dan transparan bagi proses yang dieksekusi oleh sistem.

7.3.2.6 Proses Translasi Alamat Logika ke Alamat Fisik

Proses translasi alamat logika ke alamat fisik adalah mekanisme pada sistem operasi yang menerapkan teknik paging untuk mengonversi alamat yang dihasilkan oleh CPU menjadi alamat yang valid di memori utama. Proses ini dimulai ketika CPU menghasilkan alamat logika yang terdiri atas *page number* dan *offset*. Alamat logika tersebut selanjutnya diteruskan ke *Memory Management Unit* (MMU) untuk dilakukan proses pemetaan. MMU menggunakan *page number* sebagai indeks untuk mengakses *page table*, atau *Translation Lookaside Buffer* (TLB) apabila tersedia, guna memperoleh *frame number* yang sesuai di memori utama. Setelah *frame number* diperoleh, MMU menggabungkannya dengan *offset* yang nilainya tetap, sehingga terbentuk alamat fisik. Alamat fisik inilah yang digunakan untuk mengakses data pada memori utama. Apabila halaman yang diminta tidak terdapat di memori utama, maka akan terjadi *page fault*, sehingga sistem operasi harus memuat halaman tersebut dari penyimpanan sekunder. Mekanisme ini memungkinkan pemisahan antara alamat logika proses dan lokasi fisik data di memori, serta menjadi dasar bagi implementasi sistem memori virtual.



7.3.3 Contiguous Memory Allocation

Contiguous Memory Allocation adalah metode pengalokasian memori pada sistem operasi di mana setiap proses ditempatkan pada satu blok memori yang berurutan dan berkesinambungan di memori utama. Dalam mekanisme ini, seluruh bagian proses harus berada dalam satu area memori yang bersebelahan, sehingga alamat logika proses dapat dengan mudah dipetakan ke alamat fisik menggunakan satu nilai dasar (*base address*). Pendekatan ini sederhana dan mudah diimplementasikan, serta memiliki overhead pengelolaan yang rendah. Namun, contiguous memory allocation memiliki keterbatasan, yaitu dapat menimbulkan *external fragmentation*, yaitu kondisi ketika terdapat ruang kosong di memori tetapi tidak cukup besar untuk menampung proses baru. Selain itu, metode ini kurang fleksibel dalam mendukung

multiprogramming tingkat tinggi dan tidak cocok untuk sistem dengan kebutuhan memori yang dinamis.

7.3.4 Fragmentasi dan Kompaksi

Fragmentasi adalah kondisi pada sistem manajemen memori dimana sebagian ruang memori tidak dapat dimanfaatkan secara optimal akibat pola alokasi dan dealokasi proses. Fragmentasi dibedakan menjadi dua jenis, yaitu *fragmentasi internal* dan *fragmentasi eksternal*. Fragmentasi internal terjadi ketika ruang memori yang dialokasikan lebih besar daripada kebutuhan proses, sehingga terdapat sisa ruang di dalam blok memori yang tidak terpakai. Sementara itu, fragmentasi eksternal terjadi ketika terdapat banyak ruang kosong di memori utama, tetapi ruang tersebut terpecah-pecah dan tidak bersebelahan, sehingga tidak dapat digunakan untuk menampung proses baru yang membutuhkan ruang memori besar.

Kompaksi adalah teknik yang digunakan untuk mengatasi fragmentasi eksternal dengan cara memindahkan proses-proses yang berada di memori utama sehingga ruang kosong dapat digabungkan menjadi satu blok memori yang utuh dan berkesinambungan. Dengan kompaksi, sistem operasi dapat menyediakan ruang memori yang lebih besar untuk proses baru. Namun, proses kompaksi memerlukan overhead tambahan karena melibatkan pemindahan data dan penyesuaian alamat, sehingga tidak selalu diterapkan secara terus-menerus dan biasanya dilakukan pada kondisi tertentu.

